ZSV: Zcash Stablecoin Vault

A narrow native protocol for a ZEC-collateralized dollar target

Giacomo Barbieri

5 November 2026 · research draft 0.5

Contents

1 Abstract **2**

2 Motivation and boundaries **2**

3 Design alternatives **2**

4 Implemented model and proposed native system **3**

5 Asset restriction and selective ZSA reuse **4**

6 Vault state and authorization **5**

7 Parameters and monetary arithmetic **6**

8 State transitions **6**

9 Liquidation rounding and bad debt **7**

10 Oracle consensus and trust **7**

11 Dollar target and economic limitations **8**
11.1 Funding and stabilization study 8
11.2 Bounded inventory reserve 9
11.3 Reserve shutdown and allocation limits 9
11.4 Combined vault and reserve stress 10

12 Privacy **10**

13 Zebra implementation proposal **11**

14 Strengths, costs and unresolved risks **11**

15 Research roadmap and release gates **12**

16 References **12**

1 Abstract

Zcash Stablecoin Vault (ZSV) is a proposed consensus extension for locking native ZEC and issuing one dollar-targeting stablecoin on Zcash. ZEC and a single protocol-defined stable asset are the complete asset set. The protocol excludes arbitrary issuers, token descriptions, asset registration transactions, NFTs, liquidity tokens, governance tokens and programmable asset factories.

ZSV couples stablecoin issuance to collateralized debt, repayment to token destruction, and liquidation to atomic exchange of debt tokens for collateral. Upgraded Zcash full nodes would verify the proposed authorization and state-transition rules. The repository does not yet implement that native verification. Price observations remain an external trust input, but their acceptance, freshness and use are consensus rules. No external chain, bridge, custody committee or application server determines whether a vault can spend its collateral.

The repository provides a regtest-only Rust account-ledger model, signed requests and oracle observations, economic stress tests, adversarial review and offline profiles linked to a pinned Zebra library. The first profile tests block commitments and branch replay. A second verifies serialized transparent spends against fixture UTXOs, native signatures and an experimental vault guard. Node-verifier integration, a native stablecoin value pool and shielded stablecoin transfers remain unimplemented. The model has no proven dollar peg. The protocol is original design work; cited Zcash proposals supply context and do not imply approval.

2 Motivation and boundaries

ZEC holders may want dollar-denominated payment liquidity without selling all of their ZEC. A collateralized debt protocol can supply that liquidity by issuing debt tokens against overcollateralized vaults. The holder retains upside exposure to the remaining collateral and accepts liquidation risk. Stablecoin holders instead acquire exposure to collateral valuation, liquidation execution, oracle honesty and protocol solvency.

General-purpose Zcash Shielded Assets (ZSAs) would support many assets, including speculative tokens. ZSV permits one additional monetary instrument while withholding a general token-launch interface. A smaller asset surface can reduce registry state, issuer authorization cases, wallet discovery complexity and some misuse opportunities. Financial, privacy and consensus risks remain.

ZSV prohibits arbitrary native asset issuance through its own rules. The protocol cannot prevent off-chain IOUs, inscriptions, wrapped derivatives, external-chain tokens, speculative trading or future Zcash upgrades. It cannot determine the social meaning of a payment.

3 Design alternatives

The selected route is a dedicated native vault bundle with a fixed state machine. This has a larger consensus-specific implementation than an ordinary wallet application, but its permitted behavior can be enumerated and reviewed.

A general proof-verification or predicate layer is another possible route. It could support private applications and flexible computation, but requires rules for program identity, permitted issuance, execution limits and data availability. If deployed openly, it could recreate general-purpose issuance. ZSV does not need that surface for its first experiment.

External smart-contract execution and threshold custody are excluded because the requirement is native Zcash enforcement. An off-chain service may propose a transaction or generate a proof; it must never gain discretion to make an otherwise invalid transaction valid.

ZIP 222 describes consensus-distributed verification of complex transparent spending preconditions [S1]. In June 2026, upstream described TZE as experimental, development-only and not planned for deployment [S2]. More recent work includes an extensible transaction format [S3] and research on private programmability [S7].

Veylor is one example of dollar access alongside shielded ZEC: its swaps deliver USDC held in Railgun on Ethereum [S10].

4 Implemented model and proposed native system

The executable prototype uses an account ledger to make balances, authorization and invariants easy to inspect. Its state includes simulated free ZEC, stable balances, vault collateral and principal, account nonces, the accepted oracle observation, outstanding supply and explicit bad debt. Optional reserve state adds cash, existing token inventory, funding and trading counters, an NSM accounting bucket and shutdown records. Genesis fixtures and an immutable regtest funding schedule supply simulated ZEC. Real Ed25519 signatures authenticate model requests. The collateral profile separately verifies a restricted native transparent signature format; shielded spend proofs remain future work.

The native proposal requires transaction encoding, a designated vault value path, branch-specific contextual state, correct Zcash digests, wallet construction and atomic node state commits. The account ledger cannot replace Zebra's UTXO and shielded-note accounting. The integration document identifies upstream files and acceptance gates against a pinned Zebra commit [S8].

The separate Zebra-linked runner commits a canonical signed model payload and its parent context in a zero-valued coinbase output, checks the native transaction Merkle root and follows native block hashes from regtest genesis. Replaying a branch re-executes model validation. This is an offline data-binding experiment: the output does not encumber ZEC, create native zUSD or make existing nodes enforce vault rules. Fixtures are parsed native block structures without mining or full consensus verification. Durable node state and consensus fork choice remain future work.

The collateral profile adds serialized V4 transparent transactions and branch-specific UTXOs. Its custom bootstrap block contains a funding coinbase whose outputs exactly match the model's initial ZEC balances. The block has a computed Merkle root and hash, but funding is trusted fixture issuance: subsidy, maturity and proof of work are not validated. The bootstrap uses native height 1 and model height 0; subsequent native heights are one greater than model heights. It is not the standard regtest genesis block.

Every native user transaction must pair with one signed model request. Wallet inputs use compressed secp256k1 P2PK keys, strict DER, low-S signatures and `SIGHASH_ALL` over actual branch prevouts in a fixed historical Sapling context. User transactions require a finite positive expiry height; zero expiry is rejected. Fixture configuration explicitly binds each Ed25519 model owner to its distinct native key. A separate Ed25519 intent binds the exact unsigned native transaction and signed model request to the profile domain and parent checkpoint. The domain includes the complete bootstrap block, owner bindings and model genesis commitment.

The profile compares native UTXO totals with each wallet's free ZEC and each vault's collateral after every transaction. Every vault input and output must name a vault touched by that authorized

request, including replacements that leave its balance unchanged. Failed blocks commit neither model nor UTXO changes. The experimental vault script ends in `OP_TRUE`; unmodified Zcash nodes do not enforce its restriction. All protection comes from this profile's contextual guard. zUSD remains a signed model balance. Reserve-enabled state, synthetic funding, shielded paths and fees are excluded until their native accounting is specified.

The regtest model has no peer-to-peer service, miner, payment endpoint or real-fund deposit address. It implements public vault data and public stable balances. Shielded stablecoin payments are a separate future component.

The primary proposal is now the unnumbered Consensus ZIP candidate, `zips/draft-ijaack-zsv.md`, owned by Giacomo Barbieri. Its initial scope is public payments; shielding is follow-up work. It specifies a new transaction format, a dedicated collateral value pool, fixed-key zUSD outputs, a 5-of-7 anchored oracle with twelve-block validity and normalized vault transitions. All operations in a block use its parent's committed price observation; a new report takes effect from the following block. This prevents same-block report renewal from invalidating pending operations, at the cost of one block of additional price delay. These are proposed native rules beyond the tested model. The candidate also permits holders to retire recorded bad debt by burning existing tokens without compensation; only an actual reduction of bad debt to zero resumes borrowing. No recovery incentive or dollar-redemption guarantee follows. Dev Fund allocation and reserve stabilization remain separate proposals.

The candidate includes financial, unsigned-wire and canonical-oracle fixtures. They do not replace native implementation, complete signed/block vectors, economic calibration or qualified independent consensus, cryptographic and privacy review. ZIP assignment and activation remain separate steps; the unnumbered Draft is submitted in zcash/zips#1438.

5 Asset restriction and selective ZSA reuse

The native design recognizes exactly two assets: ZEC and a protocol-defined stablecoin provisionally called zUSD. No user-selected identifier, asset description, issuer key or mint authority is accepted. Its target unit is the US dollar; replacing it with a different instrument is a protocol change.

ZIPs 226 and 227 propose general shielded asset transfer/burn and issuer-authorized creation [S4,S5]. ZSV can investigate reusing:

- Per-asset conservation and value-commitment techniques.
- Asset-aware note encryption and wallet scanning.
- Reviewed arithmetic and circuit gadgets appropriate for two fixed assets.
- Atomic multi-asset value flows, where compatible with the restricted design.

ZSV excludes generic issuance bundles, arbitrary AssetIds, arbitrary issuer-defined assets, asset metadata registries, supply-finalization controls and NFT modes. Only a validated vault transition authorizes stablecoin creation; an issuer signature cannot authorize it.

At the wire-format and circuit level, every place that introduces an asset tag must constrain it to the two-element set. Restricting a wallet menu is insufficient. Asset equality, value conservation and stablecoin mint/burn authorization must remain bound throughout proof verification and contextual checks. Any reuse creates a new protocol profile requiring independent analysis; existing ZSA audits are insufficient to establish ZSV's security.

QEDIT describes ZSAs as feature-testnet software requiring community mainnet activation [S6].

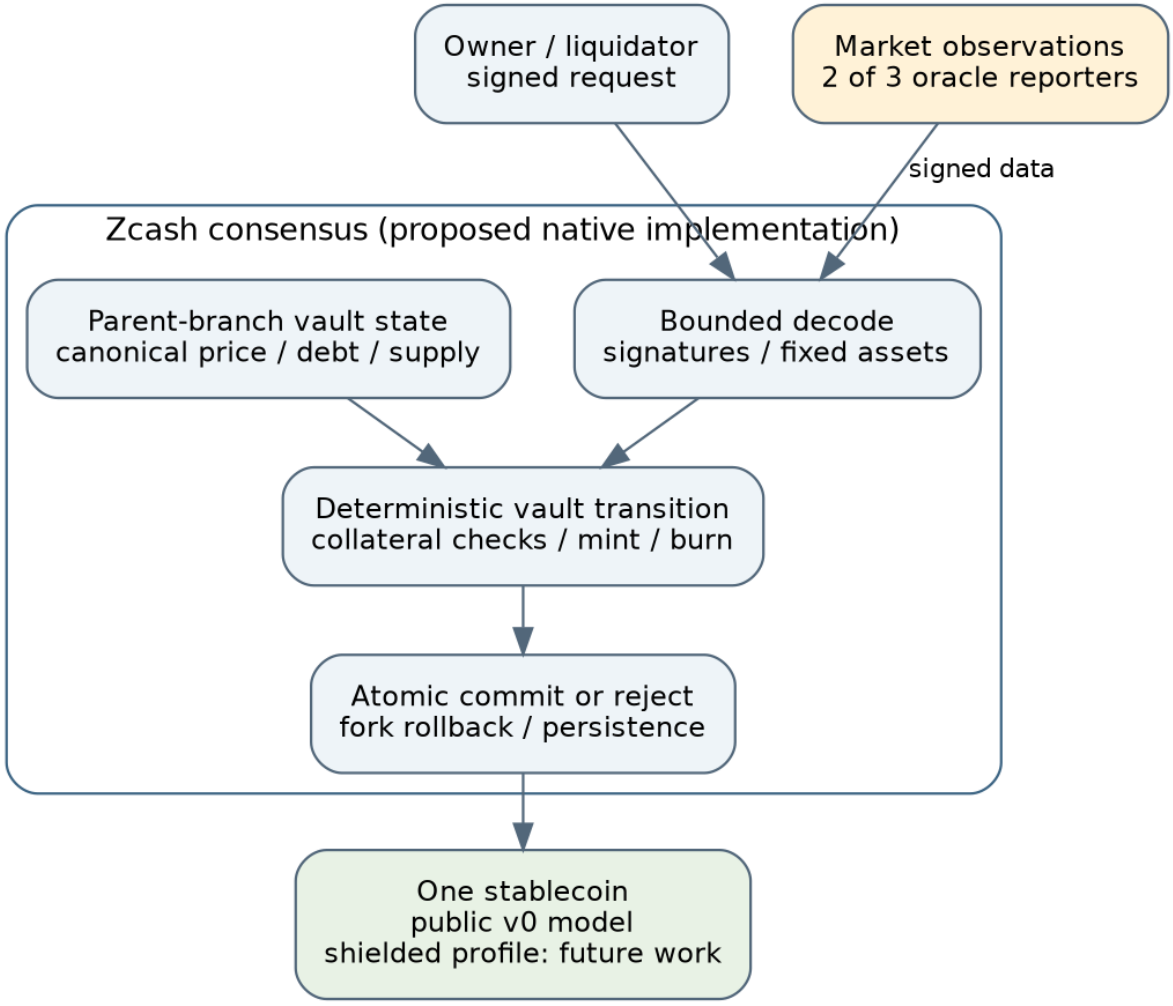


Figure 1: Proposed native consensus flow. The executable account model tests the transition rules; it does not implement this complete node pipeline.

6 Vault state and authorization

A proposed native vault stores its owner authorization, collateral C, principal D, version and data needed for any future fee calculation. Vault state must have a unique current instance. A transition consumes that instance and creates exactly one valid successor or closes it. Owner authorization is necessary for borrowing and withdrawing but insufficient to override collateral rules.

Collateral cannot remain spendable by an ordinary payment signature, an issuer key or an alternate transaction type. The node must recognize the vault encumbrance throughout locking, successor creation and release. Duplicate successors, replayed spends, cross-network authorizations and reuse of a stablecoin burn must fail.

The reference model signs an explicit action encoding and nonce. Both transaction and oracle

signatures bind an immutable model-chain domain derived from the canonical regtest configuration and genesis allocations. Reserve-enabled chains also bind all reserve parameters, the initial donor and its contribution. Vault IDs are scoped by owner public key and a local integer; a different owner cannot reserve that namespace by opening and closing the same local number. Identical canonical configuration and genesis allocations deliberately represent the same chain, including its forks. This binary authorization format is experimental and specific to the model. A native implementation must adopt an approved Zcash transaction version, consensus branch domain and appropriate digest rules.

7 Parameters and monetary arithmetic

The model uses zatoshi for ZEC and micro-units for zUSD. One ZEC equals 100,000,000 zatoshi, and one zUSD equals 1,000,000 micro-units. Monetary computations use checked integers; consensus must reject overflow rather than use wrapping, saturation or floating-point comparisons.

Illustrative parameters are:

Parameter	Model value
Borrowing / withdrawal collateral ratio	250%
Liquidation threshold	strictly below 200%
Liquidator collateral bonus	5%
Minimum nonzero vault debt	100 zUSD
Global outstanding-debt ceiling	100,000 zUSD
Interest / stability fee	zero

These values are not market-calibrated recommendations. The 250% opening margin reaches 200% after a 20% decline in collateral price. Block delays, oracle delays, market gaps and execution slippage consume that margin.

Let P be accepted micro-USD per ZEC and q=100,000,000. To avoid fractional-price ambiguity, collateral-ratio comparisons multiply integer numerators before division, subject to explicit bounds. A vault with zero debt does not require a price to release its collateral.

8 State transitions

Open and deposit. An owner creates a vault or adds actual ZEC to it. Adding collateral does not require a fresh price and does not mint stablecoins.

Borrow. Owner authorization, fresh accepted price, minimum-debt and global-cap checks are required. After adding principal, the vault must satisfy 250% collateralization. The corresponding stablecoin amount is credited in the same atomic transition. Any recorded bad debt disables further borrowing.

Repay. Stablecoins are removed from the payer and principal decreases by exactly the amount burned. The minimum remaining debt rule applies unless principal becomes zero. A stale oracle must not prevent repayment.

Withdraw and close. Owner authorization is required. Withdrawal against debt needs a fresh price and the 250% ratio. Debt-free withdrawal does not depend on oracle availability. Closure returns remaining collateral only once principal is zero.

Transfer. Existing stablecoin balances can move between authenticated accounts in the model. Native payment implementation is separately specified. Transfer cannot create supply, authorize a new asset or change debt.

Liquidate. A fresh canonical price establishes that the vault is strictly below 200%. A third party supplies stablecoins and receives the permitted ZEC in one atomic operation. The owner cannot veto it. The signed liquidator request binds the canonical oracle sequence, maximum stablecoin burn and minimum collateral received, so a queued request cannot silently execute at a different oracle epoch or outside its bounds. The model uses full vault closure to avoid a larger partial-liquidation design surface.

9 Liquidation rounding and bad debt

For bonus ratio 105/100, define the maximum affordable principal burn:

```
x = min(D, floor(C * P * 100 / (q * 105)))
```

The collateral delivered to the liquidator is:

```
z = ceil(x * q * 105 / (P * 100))
```

The verifier establishes z<=C. It burns x stable units, transfers z zatoshi to the liquidator, returns C-z to the owner and removes the vault. Remaining liability D-x becomes explicit bad debt B. Affordability rounds down and collateral payment rounds up.

If x<D, residual owner collateral has oracle value below 1.05 micro-USD under these formulas. If x=0, a dust close can be valid but has no economic incentive, so it may never be submitted.

The accounting invariants are:

```
outstanding stable supply = sum(live vault principal) + bad debt
```

and:

```
sum(wallet stable balances) + reserve inventory = outstanding stable supply
```

During liquidation supply falls by x while live debt falls by D and bad debt rises by D-x. Removing the bad-debt entry would conceal outstanding liabilities. Wallet ZEC, vault collateral, reserve cash and the NSM bucket together equal genesis ZEC plus cumulative scheduled fixture funding. Without the optional reserve, no block funding is created and the original conservation rule applies.

New borrowing freezes when B>0. A tiny shortfall can therefore halt all new borrowing, creating a griefing risk. Reserve closure burns its own inventory against recorded bad debt and can clear only that covered amount. Remaining losses keep borrowing frozen. A production design still needs a broader loss-allocation and recapitalization policy.

10 Oracle consensus and trust

The model accepts matching observations from at least two distinct authorized keys in a three-reporter configuration. Reports bind their sequence, price and expiry. The verifier enforces positive

integer prices, signature validity, reporter uniqueness, monotonic sequence and a bounded acceptance lifetime. Users cannot attach a convenient older report to override the canonical accepted state. Reserve quotes additionally limit the accepted report's age since inclusion. The signed format has no market-observation timestamp; a quorum can publish old data in a new report and pass those checks.

Consensus must use deterministic chain information, not a node's wall clock or an HTTP call to an exchange. If multiple observations are possible in a block, ordering and freshness semantics must be specified so different nodes produce the same result. Reorganizations must roll back oracle state with vault and monetary state.

Reporter honesty and independence remain trust assumptions. In the three-reporter model, two compromised reporters can inflate collateral values to authorize excess borrowing or depress prices to enable wrongful liquidation. Distinct signing keys do not establish independent organizations or price sources. A stale-policy halt trades oracle safety for borrower/holder liveness. The candidate instead requires five of seven reporters and fixes membership by network upgrade. Actual operators, source independence and recovery arrangements still require review.

11 Dollar target and economic limitations

Vault repayment creates demand for stablecoins, and borrowing creates supply when collateral capacity and incentives permit. These channels can help a dollar target but do not establish an unconditional redemption floor or ceiling. The model implements neither a holder right to redeem at one dollar of ZEC nor a monetary policy that guarantees a price of one dollar.

Mint capacity is capital constrained, bad-debt freezes can shut it off, and borrowers need not repay quickly when stablecoins trade below target. A stablecoin may therefore trade away from one dollar even while every model invariant holds.

The 5% liquidation bonus is paid out of collateral. Full debt coverage requires collateral oracle value of at least 105% of principal, before real-world execution costs. An oracle-price crash can produce bad debt even with immediate honest execution; a market price worse than the oracle can make a nominally profitable liquidation unattractive.

A production proposal needs credible demand, liquidation depth, stress-tested caps, peg incentives, recapitalization and emergency settlement. Those decisions must be tested against adversarial scenarios rather than inferred from high initial collateralization. The coding-agent economic review supplies concrete counterexamples and additional acceptance scenarios.

11.1 Funding and stabilization study

A separate mathematical study tests diverting a fraction of proposed ZCG funding reductions to a bounded ZEC incentive reserve. The forum proposal's rounded future-funding comparison implies about 62,000 ZEC less funding; a hypothetical 10% diversion would be about 6,200 ZEC over its horizon, excluding the separate treasury donation. This is an aggregate scenario, not an approved or upfront balance. Current grant streams expire around 2028, whereas the taper discussion extends reduced funding to 2032. An exact payable stream therefore needs an explicit reference and extension budget.

The study proves conditional stability bounds for an assumed linear market response. With no delay, gain k must lie between zero and two; with one epoch of delay it must lie between zero and one.

Finite reserves, limited collateral and borrowing freezes can prevent peg recovery even within those bounds. A reserve held in ZEC shares the collateral's drawdown risk.

The adversarial test rejects raw mint/burn bounties: a borrower can mint during one rewarded epoch, retain the tokens and burn them during a later rewarded epoch, collecting both payments without an external trade. Age requirements and epoch netting do not prove market impact. The Python study includes 10 finite-budget scenarios and a 240-case parameter grid; inputs are uncalibrated. It changes no Rust monetary rules. Equations, sources, counterexamples and reproducible results are in the mathematical report.

11.2 Bounded inventory reserve

The inventory experiment exchanges existing ZEC and zUSD at standing target-centred bid/ask quotes. Purchased tokens remain reserve inventory; sales require existing inventory. These transfers leave total supply and liabilities unchanged. Only the accepted ZEC/USD conversion report is needed. Its first Python version used gross per-epoch and lifetime outflow limits, order bounds, freshness and a sunset. The signed Rust version retains epoch limits and replaces lifetime trading stops with the funding cap described below. Excess new funding goes to an NSM accounting bucket; trade proceeds remain available for future purchases.

In an assumed linear market, sufficient capital and arbitrage move an initial 0.95 or 1.05 dollar price to the illustrative 0.995–1.005 dollar corridor. Buying 4,500 units costs approximately 4,477.50 dollars; initial inventory has its own acquisition cost. Eleven scenarios and 144 parameter combinations test inventory and cash constraints. These inputs are uncalibrated and omit endogenous vault issuance and liquidation.

The spread makes unchanged-price exact round trips costly. Stale or compromised ZEC conversion can still transfer substantial value, fundamental token losses can drain the bid, and losing cycles can occupy scarce spending allowances. Review reproduced and corrected a working-capital drain caused by sweeping trade proceeds prematurely. The reserve report records those Python experiments. The Rust integration now shares balances and liabilities with actual model vault issuance; it does not establish the market-response assumptions of the Python study.

11.3 Reserve shutdown and allocation limits

The lifecycle policy caps initial reserve ZEC plus later retained funding credits. Sales retain working capital without restoring funding headroom. Gross per-epoch limits and a sunset remain. The 20-epoch cycle that previously occupied the lifetime purchase allowance can therefore be followed by another valid purchase when cash and epoch capacity remain.

Normal quotes pause when the accepted report exceeds the inclusion-age or expiry limit and close after a fixed uninterrupted outage or at sunset. An elapsed deadline closes trading before a new report can resume it. Closure burns held tokens against explicit bad debt once, returns cash to the NSM bucket and offers residual token inventory through an oracle-free descending-price auction. The burn precedes user transactions; later liquidation losses remain uncovered. Active principal and collateral remain unchanged. Review of the Python policy removed a free terminal distribution that enabled borrowing solely to collect its payout and fixed stale observations postponing closure indefinitely.

The signed Rust implementation begins with zero reserve tokens and debits initial cash from a named genesis donor. Vault borrowing supplies the tokens that users may later sell to the reserve.

Every reserve order signs its direction, quantity, account nonce, report sequence, epoch, expiry and slippage bound. Auction orders sign quantity, expiry and maximum ZEC input. Additional funding comes only from the immutable regtest schedule, once per block, and defaults to zero. Funding, fills, shutdown, inventory burns and vault actions commit or fail together, and the existing contextual adapter prepares the combined state against an exact parent checkpoint.

Returning cash to NSM can leave uncovered bad debt; auction competition and discrete price steps can produce poor proceeds. Unsold inventory stays available at a positive floor, so liquidation is not guaranteed. The lifecycle report records these choices and exact accounting cases, while the Rust specification defines their signed implementation. Native coinbase funding, NSM subsidy rules and persistent Zebra branch state remain proposed. No Zcash allocation has been approved for this experiment.

11.4 Combined vault and reserve stress

The signed-ledger study runs 351 deterministic cases: 324 combinations of ZEC price, oracle lag, liquidator token balance, reserve inventory and liquidation timing; six stale-price trade cases; 12 funding cases; and nine auction cases. Every reserve token comes from signed vault issuance. The study records actual ledger losses separately from collateral shortfalls in active vaults. Its post-crash prices remain constant, so delayed reports can produce the same final balances while extending the period before settlement. These cases are conditional calculations, not independent markets or forecast probabilities.

A 25-ZEC vault owing 1,000 zUSD creates 523.809524 zUSD of bad debt when liquidated at 20 dollars per ZEC. With 800 reserve tokens, liquidation before closure lets the terminal burn cover that loss. The same liquidation in the closing block runs after the one-time burn, leaving all 523.809524 as bad debt while 800 tokens remain in the auction. With insufficient liquidator tokens at 10 dollars per ZEC, liquidation fails and recorded bad debt stays zero, although active principal exceeds collateral value by 750 dollars.

Auction participation changes the outcome. With only the borrower bidding at the default auction's floor, 400 tokens cost 400 zatoshi. The borrower can then repay and close, earning a gross completed-cycle gain of 3.979996 ZEC from its earlier reserve sale. This calculation excludes fees, interest and capital opportunity cost. With no bidders, all 400 tokens remain unsold. These outcomes justify further loss-settlement and auction design work before choosing production parameters. The combined report supplies assumptions, exact results and reproduction commands.

12 Privacy

The executable model exposes ownership, collateral, principal, oracle prices and stable balances so its transitions can be inspected. Public liquidations can reveal financial distress and enable transaction-ordering strategies.

A later design can seek shielded stablecoin payments while retaining public vault health. Even then, mint/burn amounts and timing may correlate deposits, borrowing and payments. Sharing a circuit design does not guarantee the same anonymity set as ZEC; distinguishable bundles, asset visibility at boundaries and wallet behavior matter.

A liquidator for a fully hidden vault needs the information and witnesses to construct a transaction when an uncooperative borrower disappears. A validity proof that hides correct arithmetic does not

supply those witnesses. Public liquidation information, specialized proofs, mediated computation or other designs alter privacy, availability and trust. Using Halo 2 alone does not resolve this problem.

13 Zebra implementation proposal

The source baseline is Zebra commit 5faf8a413f0d30a6e93115f3d16af1e229fd7858, dated 28 October 2026 [S8]. The integration document records actual transaction, verifier and state APIs. At that baseline Zebra wraps a librustzcash transaction representation; a new native bundle requires coordinated library work rather than adding a guessed method to Zebra alone.

The staged proposal is:

1. Define the wire format, activation domain, signatures and fixed asset encodings. Do not reuse v6, which already denotes NU6.3/Ironwood [S9]. ZIP 248 is draft future infrastructure, not deployed permission to introduce a bundle.
2. Implement bounded parsing, canonical digests and fixed asset types in the transaction library. Unknown tags, generic issuance and alternate collateral-release paths must fail closed.
3. Add structural checks and signature verification to the transaction verifier, including the mempool path. Public v1 requires no new shielded circuit. Mempool acceptance is not a substitute for parent-branch contextual verification.
4. Implement branch-specific oracle, vault, stable UTXO, supply and bad-debt state in Zebra. Collateral belongs to the dedicated vault pool; ordinary transactions cannot access it. Protect checkpoint/direct-finalization paths as well as ordinary contextual verification. Commit ZEC and ZSV changes atomically, preserve fee accounting and undo changes on a fork switch. Reserve and funding state are outside this core.
5. Add wallet construction/scanning and independently tested adversarial vectors. Introduce any fixed-two-asset shielded profile as its own reviewable stage.
6. Run differential node tests and a private development network before seeking consensus adoption. No production acceptance bit is enabled by this repository.

14 Strengths, costs and unresolved risks

Strength	Corresponding cost or weakness
Fixed native asset set	Fewer applications; derivatives/speculation remain possible elsewhere
No custody or issuer spending key	Consensus logic and its bugs become the enforcement boundary
Atomic liquidation	Requires liquidators, stable liquidity and inclusion during stress
Explicit bad-debt accounting	Does not recapitalize losses; zero-tolerance freeze can be griefed
Public v0 vault health	Exposes financial positions and transaction-ordering opportunities
Selective ZSA technique reuse	New circuit/profile still requires review; not automatically a tiny upgrade
Native price acceptance rules	Dollar information still relies on reporters and governance

Strength	Corresponding cost or weakness
Signed reserve trades and exact funding conservation	Quotas can be occupied; oracle errors and thin auctions can transfer value to traders
Native fixture UTXOs reconciled after each model transaction	Experimental guard protects only the offline profile; zUSD and reserve value paths still need native integration

Consensus expansion can harm ZEC users who never borrow. State growth, verification costs, malformed inputs, expensive signatures and inconsistent branch rollback are base-layer risks. A narrow use case is not evidence that implementation risk is small. Resource metering, state bounds and cross-implementation test vectors must be release requirements.

15 Research roadmap and release gates

The collateral profile supplies an executable check of native transparent ownership and per-transaction collateral accounting. Next work should review the ZIP candidate and implement its dedicated transaction format, collateral value pool, stablecoin accounting and durable branch state in Zebra, with independent conformance vectors and adversarial cost benchmarks. Reserve funding and shutdown remain separate research; they are not prerequisites for implementing the core ZIP.

Before any value-bearing test, require a defined oracle governance and compromise response, loss allocation, sustainable peg mechanism, source-based security audit, cryptographic review of any shielded profile, wallet safety/recovery, reproducible builds and full-node fork/rollback tests. Before mainnet, community review and an adopted network upgrade are indispensable. No mainnet date or funding estimate is asserted here.

16 References

- [S1] ZIP 222: Transparent Zcash Extensions. https://zips.z.cash/zip-0222
- [S2] librustzcash issue 2443, removal of experimental TZE support. https://github.com/zcash/librustzcash/issues/2443
- [S3] ZIP 248: Extensible Transaction Format (Draft). https://zips.z.cash/zip-0248
- [S4] ZIP 226: Transfer and Burn of Zcash Shielded Assets (Draft). https://zips.z.cash/zip-0226
- [S5] ZIP 227: Issuance of Zcash Shielded Assets (Draft). https://zips.z.cash/zip-0227
- [S6] QEDIT ZSA Hub, implementation/activation status. https://qed-it.com/zsa-hub/
- [S7] Private Programmability in Zcash, Foundation-funded research discussion. https://forum.zcashcommunity.com/t/private-programmability-in-zcash-research-results-and-community-discussion/48016
- [S8] Pinned Zebra source. https://github.com/ZcashFoundation/zebra/tree/5faf8a413f0d30a6e93115f3d16af1e229fd7858
- [S9] ZIP 229: Version 6 Transaction Format. https://zips.z.cash/zip-0229

- [S10] Veylor: split-key swap and Railgun settlement explanation, read 5 November 2026. https://veylor.cash/how-it-works/

See `sources.md` for the pinned primary-source map and `zebra-integration.md` for integration evidence. Security claims are bounded by `threatmodel.md`, `economic-review.md`, `adversarial-review.md` and the verification receipt.